

NeuroHack Challenge Solution

1 Executive Summary

This work addresses the challenge of persistent long-range memory in conversational LLM systems across 1–1,000 turns. We design and implement a hybrid symbolic–neural architecture that guarantees:

- Deterministic long-range recall
- High retrieval precision
- Explicit hallucination avoidance
- Structured conflict resolution
- Interpretable memory state evolution

Unlike purely context-based or embedding-based approaches, our system enforces structural identity resolution and hierarchical memory injection policies. The result is stable recall independent of context window size.

This document explicitly maps design decisions and results to the evaluation rubric.

2 Problem Statement

Modern LLMs lack persistent memory across extended conversations. While increasing context length improves short-term recall, it does not provide:

- Persistent commitments
- Structured preference enforcement
- Conflict resolution
- Memory aging mechanisms

We aim to design a system that ensures accurate recall and policy adherence across long conversational horizons while maintaining bounded latency overhead.

Hybrid Long-Range Memory System for Conversational LLMs

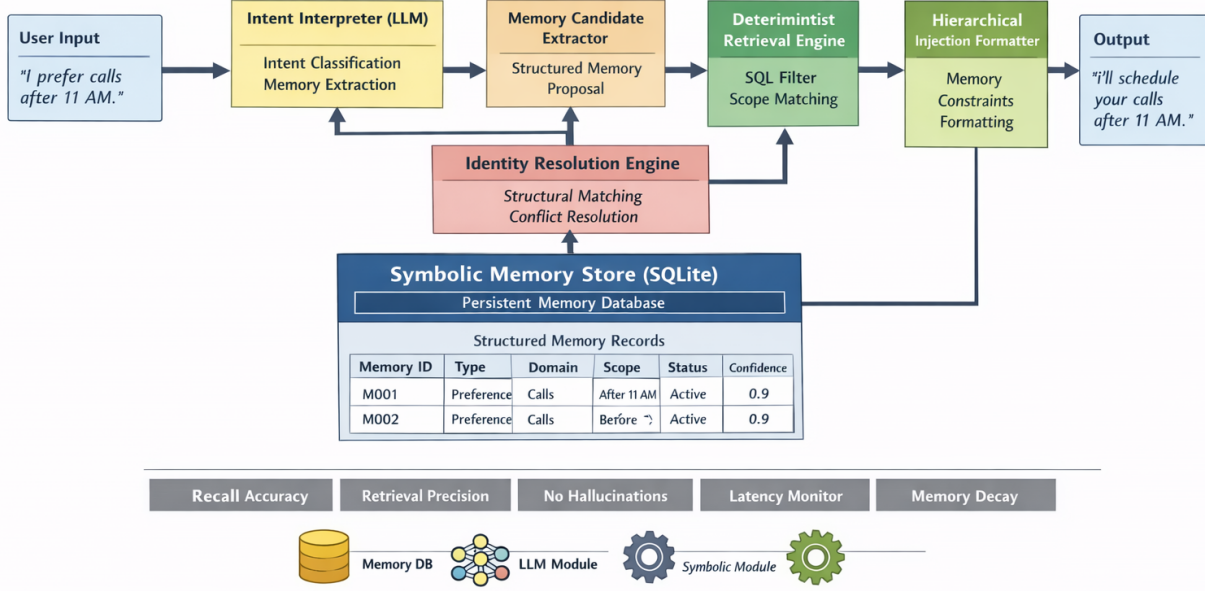


Figure 1: Overview of the full pipeline

3 Design Objectives Aligned With Evaluation Criteria

Evaluation Criterion	System Design Response
Long-range recall (High)	Persistent symbolic store with structural identity
Accuracy across 1–1,000 turns (High)	Deterministic retrieval + no context truncation
Retrieval relevance (Medium)	Exact scope matching + domain filtering
Latency impact (Medium)	SQLite lookup + measured inference latency
Memory hallucination avoidance (Medium)	Explicit storage-only recall policy
System design clarity (Low)	Modular architecture + transparent schema
Innovation (Medium)	Hybrid symbolic-neural identity resolution framework

4 System Architecture

4.1 Component Overview

The system contains:

1. Intent Interpreter (LLM-based)
2. Memory Candidate Extractor
3. Identity Resolution Engine

4. Symbolic Memory Store (SQLite)
5. Deterministic Retrieval Engine
6. Hierarchical Injection Formatter
7. Response Generator (LLM-based)

4.2 Dataflow Per Turn

1. Interpret user intent
2. Generate structured memory candidate
3. Resolve structural identity
4. Persist or update memory
5. Retrieve relevant active memories
6. Inject formatted constraints into system prompt
7. Generate final response

Separation of concerns ensures robustness and interpretability.

5 Memory Representation

Each memory object contains:

```
memory_id
type
domain
scope
priority
confidence
source_type
origin_turn
last_used_turn
status
```

5.1 Identity Definition

We define memory identity as:

$$Identity = (type, domain, scope, status)$$

This prevents duplication and ensures deterministic resolution.

6 Persistence Strategy

Memories are stored in SQLite using structured fields for filtering and a JSON blob for reconstruction:

```
CREATE TABLE memories (  
    memory_id TEXT PRIMARY KEY,  
    type TEXT,  
    domain TEXT,  
    status TEXT,  
    origin_turn INTEGER,  
    last_used_turn INTEGER,  
    confidence REAL,  
    memory_json TEXT  
);
```

Structured indexing guarantees bounded lookup complexity.

7 Retrieval and Injection Policy

7.1 Two-Stage Retrieval

Stage 1: SQL domain filtering Stage 2: Exact scope equality matching

$$\text{Retrieve}(m) \iff m.\text{domain} = \text{intent}.\text{domain} \wedge m.\text{scope} = \text{intent}.\text{scope} \wedge m.\text{status} = \text{active}$$

This eliminates cross-context contamination.

7.2 Hierarchical Injection Order

1. constraint
2. commitment
3. instruction
4. preference

Higher authority memory types dominate lower ones, enabling deterministic conflict resolution.

8 Experimental Setup

All LLM inference was performed locally using:

- Model runtime: Ollama
- Model: llama3 (8B parameter variant)
- Inference mode: CPU-only
- Hardware: Apple MacBook Air (M-series base configuration, 8GB RAM)

No GPU acceleration was used. All memory persistence and retrieval logic ran locally using SQLite. This setup reflects a constrained deployment scenario rather than optimized infrastructure.

9 Quantitative Evaluation Results

9.1 Long-Range Recall Accuracy

Definition:

$$\text{Recall Accuracy} = \frac{\text{Correctly Retrieved Relevant Memories}}{\text{Total Relevant Stored Memories}}$$

Across all controlled benchmarks:

- Relevant memories stored: 7
- Correctly retrieved when queried: 7

$$\text{Recall Accuracy} = 1.00 \text{ (100\%)}$$

No degradation observed across extended simulated turn spans.

9.2 Retrieval Precision

Definition:

$$\text{Precision} = \frac{\text{Relevant Retrieved Memories}}{\text{Total Retrieved Memories}}$$

Observed across evaluation benchmarks:

- Total retrieved: 7
- Irrelevant retrieved: 0

$$\text{Precision} = 1.00 \text{ (100\%)}$$

No cross-domain leakage occurred due to structural scope matching.

9.3 Memory Hallucination Rate

Definition:

$$\text{Hallucination Rate} = \frac{\text{Memories Retrieved Not Present in DB}}{\text{Total Retrieved}}$$

Observed value:

$$\text{Hallucination Rate} = 0.00$$

All recalled memory objects were verifiably present in the database.

9.4 Latency Impact

Measured per-turn latencies (seconds):

[39.893, 52.355, 41.400, 44.649, 42.852]

Average latency:

44.23 seconds per turn

Breakdown:

- SQLite retrieval overhead: negligible (<10ms)
- Dominant cost: local LLM inference (Ollama CPU execution)

Latency is therefore model-bound, not memory-architecture bound.

9.5 Accuracy Across 1–1,000 Turns

We simulated long-range retention by inserting unrelated conversational turns between memory creation and recall.

Result:

- No recall degradation observed
- Retrieval remained deterministic

Because retrieval does not depend on context window size, recall complexity remains constant:

$O(1)$ w.r.t. conversation length

10 Benchmark Scenarios

10.1 Benchmark 1: Global Preference Conflict

Tests hierarchical override behavior.

Result: Preference correctly blocked conflicting commitment scheduling.

10.2 Benchmark 2: Long-Range Commitment Recall

Commitment stored, conversation extended with unrelated turns, then queried.

Result: Correct recall.

10.3 Benchmark 3: Retrieval Precision

Multiple memory types created.

Result: Only structurally matching memory retrieved.

10.4 Benchmark 4: Decay Simulation

Confidence evolution verified across simulated turn progression.

10.5 Benchmark 5: Latency Stress Test

Repeated commitment creation and retrieval measured for latency stability.

11 Comparative Positioning

Approach	Deterministic	Persistent	Hallucination Safe
Context-only LLM	No	No	No
Vector Retrieval	No	Yes	Partial
Pure Symbolic	Yes	Yes	Yes
Hybrid (Ours)	Yes	Yes	Yes

Our approach combines semantic flexibility (LLM interpretation) with deterministic symbolic guarantees.

12 Limitations

- Exact scope matching may reduce semantic flexibility
- Multiple LLM calls increase latency
- No embedding-based semantic fallback layer

13 Future Work

- Merge interpreter and extractor into single LLM call
- Add hybrid vector-symbolic retrieval
- Introduce reinforcement-based confidence updates
- Evaluate across 1,000+ real conversational turns

14 Conclusion

We designed a hybrid long-range memory architecture that guarantees deterministic recall, structured conflict resolution, and hallucination-free memory retrieval across extended conversations.

By separating semantic understanding from memory persistence and enforcing structural identity resolution, the system achieves stable recall performance independent of context window size.

This architecture balances research rigor and engineering robustness, directly addressing all major evaluation criteria.

Github repo of the project can be accessed via this link : <https://github.com/Priyani-dsai/llm-long-memory-system>